Uni-FACEF
Centro Universitário Municipal de Franca

ESTRUTURA DE DADOS II - PROF. ALEXANDRE GOMES

Atividade Prática: Árvore Binária de Busca (Valor 10 - Peso 3,0)

INSTRUÇÕES:

1. Leia atentamente as instruções para realizar a atividade.
2. Conteúdo Programático: árvore binária de busca (inserção, busca e remoção), percursos em-ordem, pré-ordem e pós-ordem, e recursividade.
3. O trabalho prático deverá ser feito **INDIVIDUAL**.
4. O programa-fonte deverá ser feito em **Javascript** (módulos `.mjs`, partindo da classe `BinarySearchTree` vista em aula) e deverá ser compactado e enviado na tarefa do **AVA**.
5. A data limite para a entrega e envio do arquivo é: **08/11/2026 (23:59h)**

OBJETIVO: aplicar a árvore binária de busca (ABB) em um cadastro de alunos, usando o **RA** como chave de organização da árvore. A atividade tem duas partes: **Parte 1** (percursos e forma da árvore, valor 3,0) e **Parte 2** (cadastro de alunos, valor 7,0). Tudo é feito em código e entregue pelo AVA.

PARTE 1: PERCURSOS E FORMA DA ÁRVORE EM CÓDIGO (Valor 3,0)

Crie o arquivo `parte1.mjs`, que importa a classe `BinarySearchTree` vista em aula (`BinarySearchTree.mjs`) e executa os exercícios abaixo, exibindo na tela o resultado de cada um com um título. Cada exercício vale 1,0. Os valores são inseridos **na ordem indicada**, em uma ABB inicialmente vazia.

Exercício 1: insira `50, 30, 70, 20, 40, 60, 80` e exiba os percursos **em-ordem**, **pré-ordem** e **pós-ordem**, cada um em uma linha (por exemplo, `Em-ordem: 20, 30, 40, ...`), usando uma função de callback que guarda os valores em um array. Em um comentário no código, explique por que o em-ordem sai em ordem crescente.

Exercício 2: acrescente à classe os métodos `height()` e `countLeaves()` (a altura é contada em arestas: árvore vazia devolve -1 e árvore só com a raiz devolve 0). Insira `8, 3, 10, 1, 6, 14, 4, 7, 13` e exiba os três percursos, a altura e o número de folhas.

Exercício 3: (a) insira `1, 2, 3, 4, 5` e exiba a altura; em um comentário, explique por que essa forma é ruim para a busca. (b) insira `50, 70, 30, 60, 80, 20, 40` (os mesmos valores do Exercício 1, em outra ordem) e compare a pré-ordem com a do Exercício 1: a árvore ficou igual? (c) escolha **outra** ordem de inserção dos mesmos valores do Exercício 1 que gere uma árvore **diferente** e mostre, pela pré-ordem, que ela é diferente.

PARTE 2: PROGRAMA, CADASTRO DE ALUNOS EM ABB (Valor 7,0)

ESTRUTURA: a partir da classe `BinarySearchTree` vista em aula, construa uma ABB que armazene objetos aluno no formato `{ ra, nome, media }`. A árvore deve ser organizada pelo **RA** (compare `data.ra`, e não o objeto inteiro). Dois alunos podem ter o mesmo **NOME**, mas **nunca** o mesmo RA. Crie dois arquivos: `AlunosABB.mjs` (a classe da árvore) e `main.mjs` (o programa de testes).

Você **não** pode usar `Array.sort()` nem bibliotecas prontas de árvore.

Uni-FACEF
Centro Universitário Municipal de Franca

ESTRUTURA DE DADOS II - PROF. ALEXANDRE GOMES

REQUISITOS:

- **Inserção por RA (1,5):** adapte o `insert` para comparar pelo RA. Se o RA já existir, o aluno **não** entra na árvore: o método devolve `false` (e `true` quando insere) e o `main.mjs` exibe a mensagem "RA já cadastrado".
- **Listagem (0,5):** relatório com todos os alunos em ordem crescente de RA, mostrando todos os campos. Use o percurso em-ordem com uma função de callback.
- **Busca (1,0):** método `searchStudent(ra)` que devolve o objeto do aluno, ou `null` se não existir. O `main.mjs` deve mostrar os dados do aluno ou "RA não encontrado".
- **Remoção (1,5):** adapte o `remove` para trabalhar com o RA e teste os **três casos**: nodo folha, nodo de grau 1 e nodo de grau 2. Mostre a listagem em-ordem após cada remoção.
- **Métodos novos (1,5):** `count()` (total de alunos), `height()` e `countLeaves()` (adapte os da Parte 1) e `averageGrade()` (média das médias de todos os alunos). Dica: `count`, `height` e `countLeaves` são naturalmente recursivos, pois o resultado de um nodo depende dos resultados de seus filhos.
- **Organização e saída (1,0):** código comentado explicando cada caso, nomes claros, saída com títulos para cada etapa e, no topo de cada arquivo `.mjs` que você criar, um comentário com seu nome e RA.

DADOS PARA TESTE (fictícios), inseridos nesta ordem:

Ordem	RA	Nome	Média
1	2026050	Ana Beatriz Souza	8,5
2	2026020	João Silva	7,0
3	2026080	Mariana Lopes	9,0
4	2026010	Pedro Henrique Alves	6,5
5	2026030	João Silva	8,0
6	2026070	Camila Ferreira	7,5
7	2026090	Rafael Costa	5,5
8	2026025	Luiza Martins	9,5
9	2026075	Bruno Carvalho	6,0
10	2026095	Fernanda Ribeiro	8,0

Depois das 10 inserções, tente inserir também o aluno `2026030, Carlos Mendes, 4,0`. Essa inserção deve ser **recusada**, pois o RA já existe (note que os dois "João Silva" devem permanecer, porque têm RA diferentes).

ROTEIRO DO `main.mjs`:

- Inserir os 10 alunos e tentar inserir o RA repetido.
- Listar os alunos em ordem de RA e exibir `count`, `height`, `countLeaves` e `averageGrade`.
- Buscar o RA `2026075` (existe) e o RA `2026099` (não existe).
- Remover o RA `2026025` (folha), listar; remover o RA `2026090` (grau 1), listar; remover o RA `2026050` (grau 2, é a raiz), listar.
- Mostrar o percurso **pré-ordem** da árvore final (só os RAs) e exibir `count` e `height` novamente.

RESULTADOS ESPERADOS (para você conferir o seu programa):

```
Em-ordem (RA) após as inserções:
2026010, 2026020, 2026025, 2026030, 2026050,
2026070, 2026075, 2026080, 2026090, 2026095
count = 10   height = 3   folhas = 4   média = 7,55

Em-ordem (RA) após remover 2026025, 2026090 e 2026050:
2026010, 2026020, 2026030, 2026070, 2026075, 2026080, 2026095
Pré-ordem (RA) da árvore final:
2026070, 2026020, 2026010, 2026030, 2026080, 2026075, 2026095
count = 7   height = 2
```

BÔNUS (+0,5, a nota final não passa de 10): método `printTree()` que imprime a árvore "deitada", com a raiz à esquerda e **4 espaços de indentação por nível**, usando o percurso direita, raiz, esquerda. Exemplo para a árvore do Exercício 1:

```
        80
    70
        60
50
        40
    30
        20
```

ENTREGA NO AVA E CRITÉRIOS DE AVALIAÇÃO

Envie **um único arquivo** `.zip` chamado `ABB_NomeSobrenome.zip` contendo: `BinarySearchTree.mjs` (com os métodos que você acrescentou), `parte1.mjs`, `AlunosABB.mjs`, `main.mjs`, `saida1.txt` e `saida2.txt` (gerados com `node parte1.mjs > saida1.txt` e `node main.mjs > saida2.txt`).

Item	Valor
Parte 1: Exercício 1 (percursos)	1,0
Parte 1: Exercício 2 (altura e folhas)	1,0
Parte 1: Exercício 3 (forma da árvore e ordem de inserção)	1,0
Parte 2: inserção por RA, rejeitando repetidos	1,5
Parte 2: listagem em ordem crescente de RA	0,5
Parte 2: busca por RA	1,0
Parte 2: remoção nos três casos	1,5
Parte 2: métodos `count`, `height`, `countLeaves` e `averageGrade`	1,5
Parte 2: organização, comentários e saída	1,0
Total	10,0
Bônus: `printTree()`	+0,5

Obs: será avaliada principalmente a **lógica**. Pequenos erros de sintaxe ou de formatação da saída têm peso pequeno.

Bom Trabalho!!!
Prof. Alexandre